

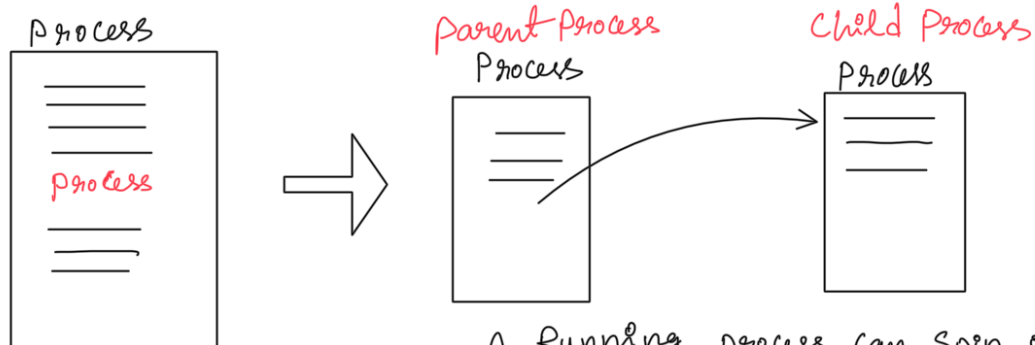
Process Thread part 2

Day 59
07/09/25

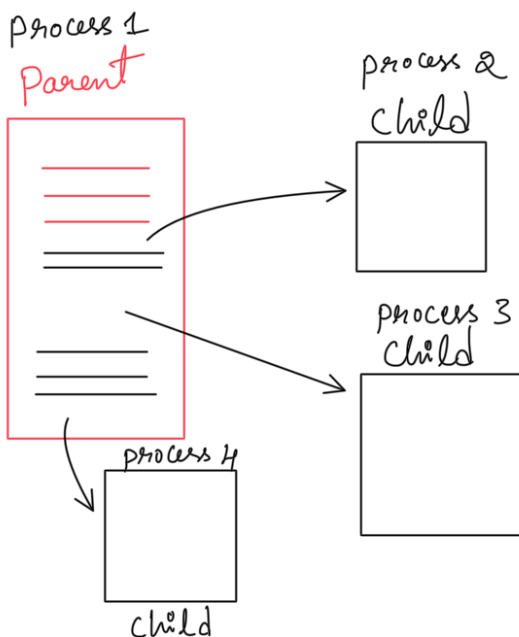
As we know till now

Process is a executable code which is loaded in the RAM for execution

Application's code will run in User mode, including Built in apps also run in user mode (default Applications)



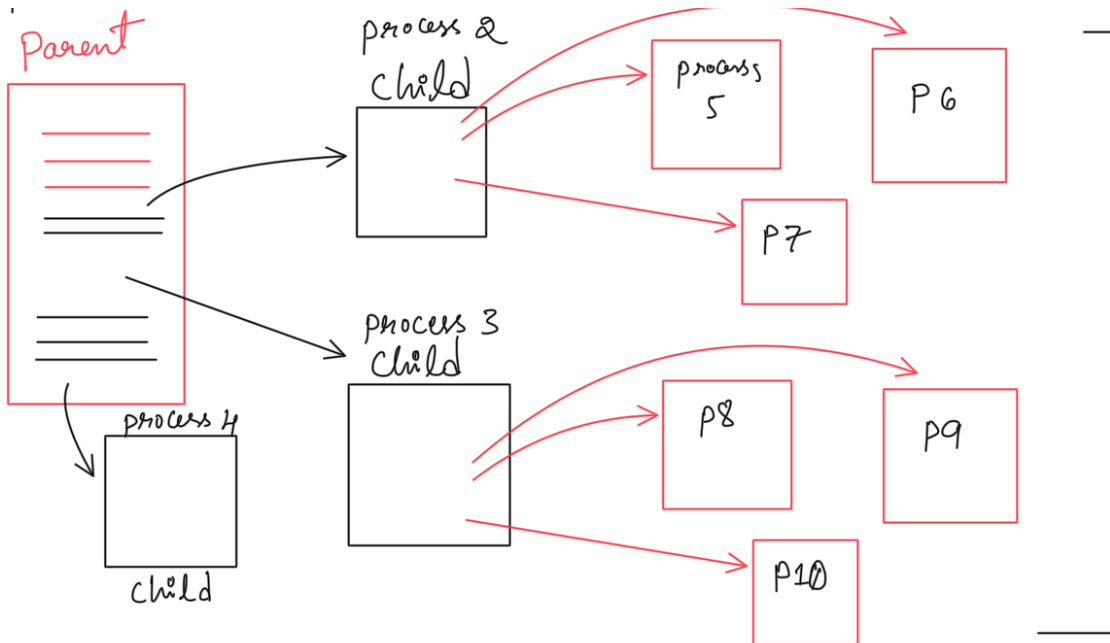
A Running process can spin off and Run another Process



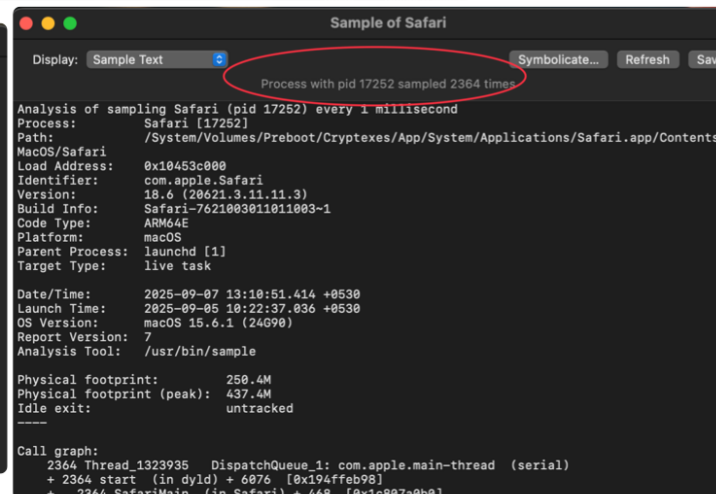
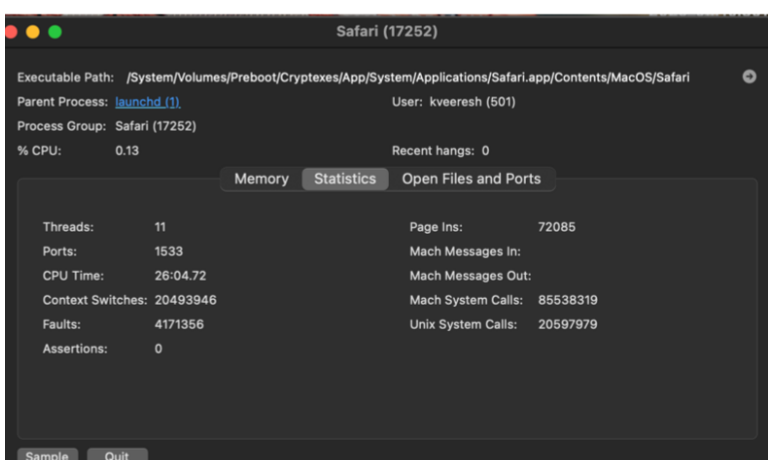
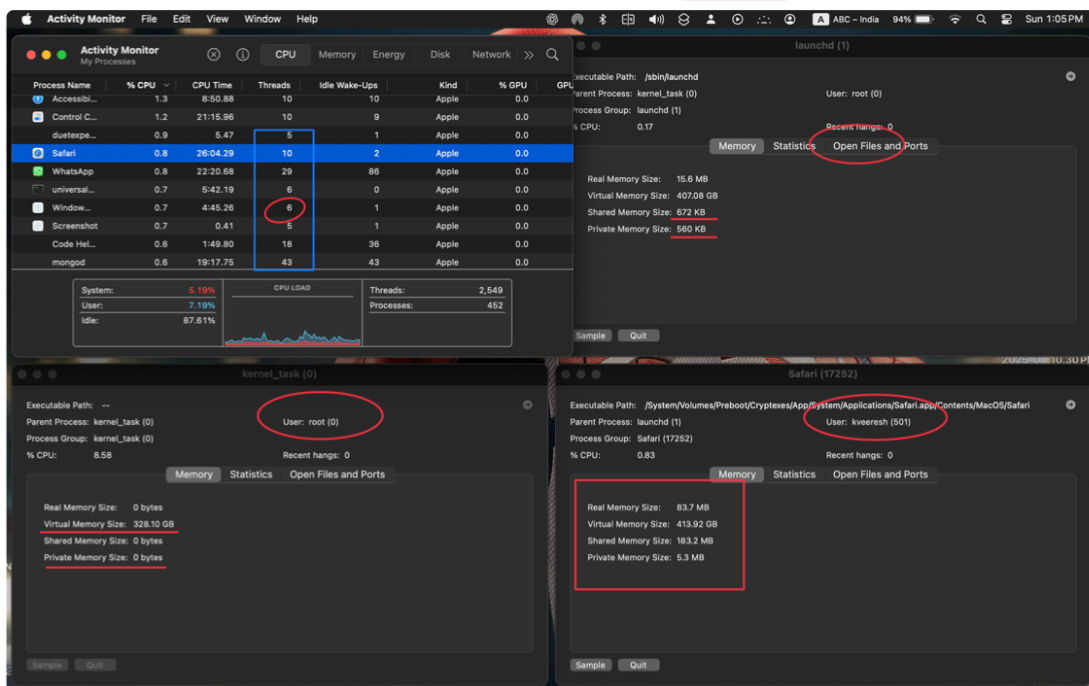
A process can create Multiple process

Including that child processes can also create other processes.

A Running process can also create a another multiple process while executing. That means a parent process which is executing can create a child process's and that child process can also further create couple of processes.

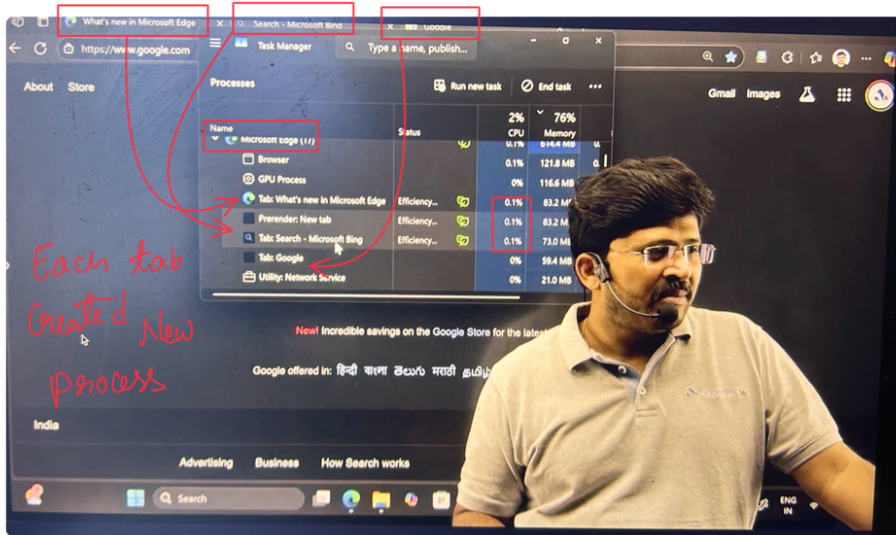


child processes can also create couple of processes.

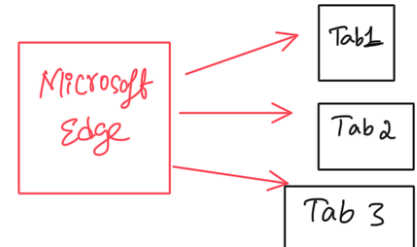


MEMORY PRESSURE		
Physical Memory:	8.00 GB	App Memory:
Memory Used:	6.51 GB	Wired Memory:
Cached Files:	1.43 GB	Compressed:
Swap Used:	1.01 GB	

Multiple Instances of Single Process

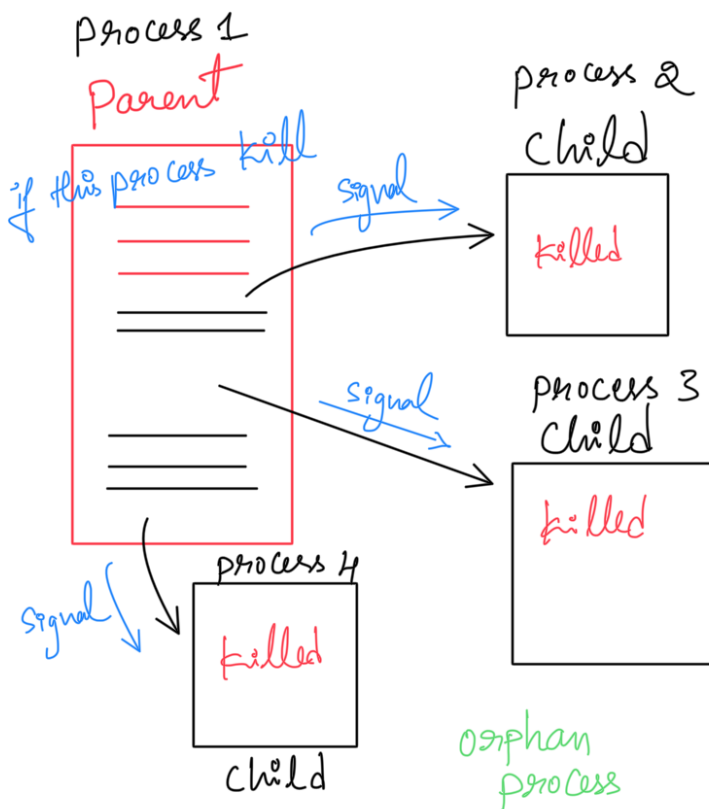


Microsoft Edge is a Parent process & Each Tab the Microsoft Edge created a New process in the CPU



If parent process got crashed child process also get crashed

If parent process closed then child process also will be terminated.



If the parent process get closed or crashed Before that a signal is sent to the child processes and they get closed.

if failed to send signal for closing in close or crash condition then the child become orphan process

1. Parent process sending signals to child

- In Unix/Linux-like OS, processes can communicate using **signals** (like software interrupts).
- A **parent process** can send signals to its child (e.g., SIGKILL, SIGSTOP, SIGUSR1).

Example in Linux shell:

```
bash
kill -SIGKILL <child_pid>
```

Example in C:

```
c
kill(child_pid, SIGTERM);
```

- If **parent crashes (or closes)**, the OS **does not automatically kill the child process** (unless the program is written that way).
- The child becomes an **orphan process** (explained below).

2. Orphan Process

An **orphan process** is a process whose **parent has terminated** but the child is still running.

👉 What happens?

- In Unix/Linux, when a parent dies, the **init process (PID 1)** adopts the orphaned child.
- **init** becomes the new parent and takes care of cleaning up when the child finishes.

💡 Example:

```
bash
$ sleep 1000 &
[1] 1234 # child PID
$ exit # parent shell exits
# Now sleep(1000) keeps running in background, adopted by init → orphan
```

So the child is still alive, even though its parent shell is gone.

Multiprocessing is Super Important

4. Real-world Scenarios

- **Parent closing/crashing but child continues**
 - A **server process** may spawn workers (child processes). If the main server crashes, workers may keep running → orphans.
- **Init (systemd in Linux) adopting orphans** ensures they don't run unmanaged.
- **Safety:** orphan adoption prevents resource leaks.

3. Zombie Process (contrast)

- A **zombie process** is different from an orphan.
- When a child finishes but the parent does not collect its **exit status** (via `wait()`), the child remains as a zombie (a "dead" process entry in the process table).

So:

- **Orphan** → parent is gone, child still alive (adopted by init).
- **Zombie** → child is dead, parent hasn't cleaned it up yet.

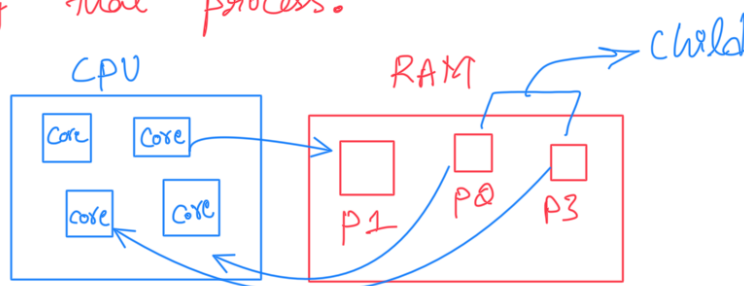
✓ Summary:

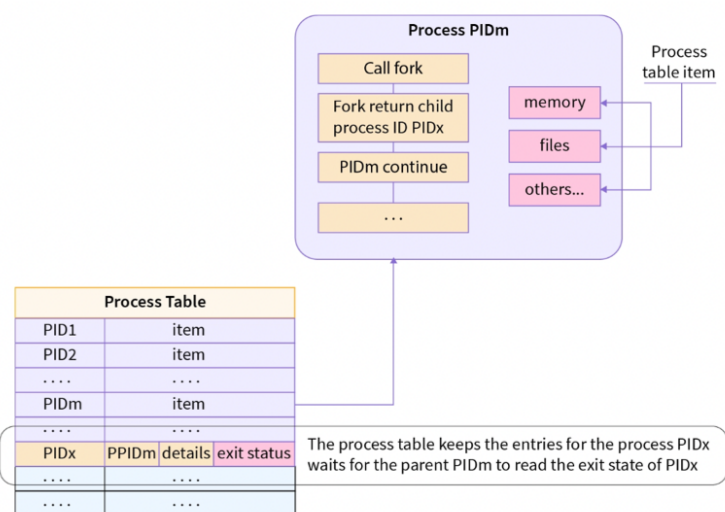
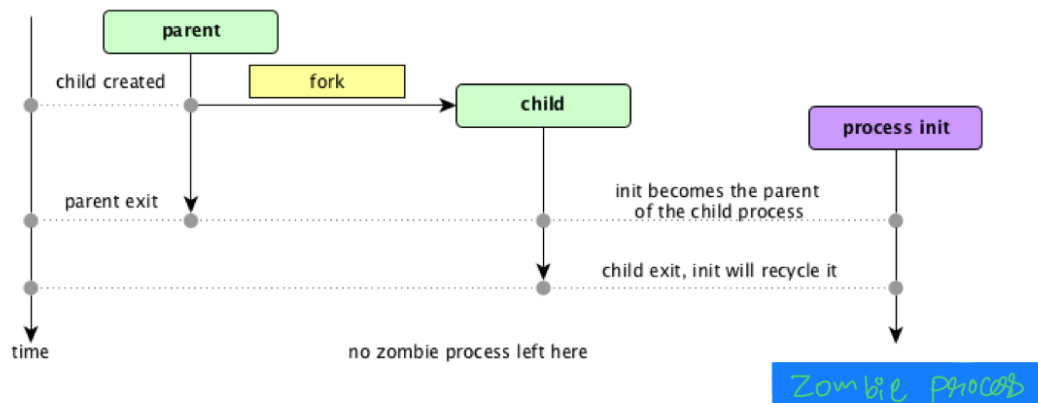
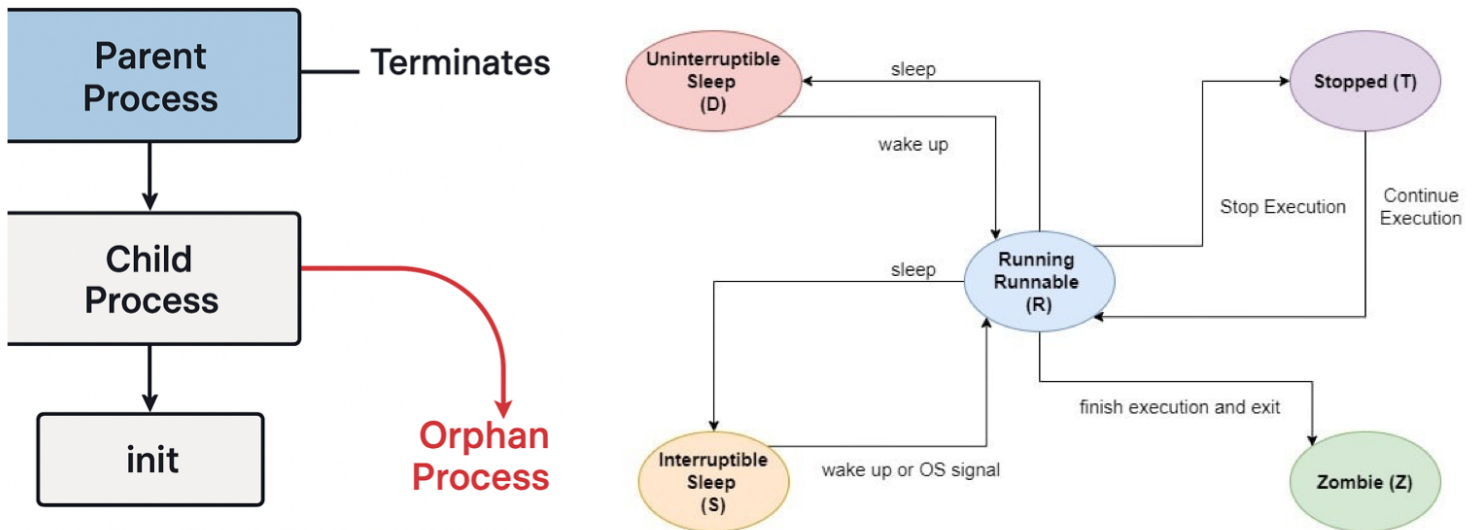
- Parent can send signals to child (kill, stop, custom).
- If parent dies, child ≠ automatically killed → becomes **orphan**.
- Orphan = child process whose parent is dead → adopted by **init (PID 1)**.
- Different from **zombie** (finished child waiting for parent cleanup).

Every Application Runs as Multiple Process

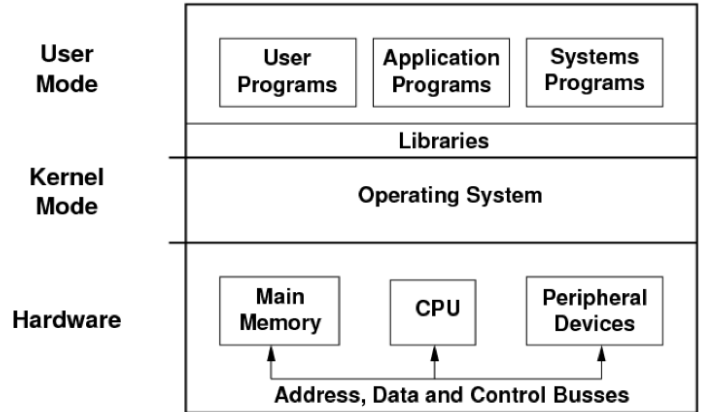
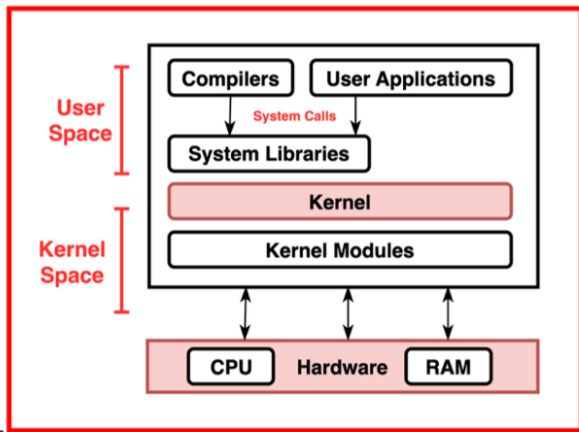
If any one portion got crashed then it doesn't effect other portion application.

Modern CPU Have Multiple Cores, So the more processes you make for single application the more chance of executing that process.





USER and kernel Mode



1. What are User Mode and Kernel Mode?

Modern CPUs provide at least **two execution modes**:

- **User Mode** → Restricted mode where applications (user programs) run.
- **Kernel Mode** → Privileged mode where the operating system kernel runs.

The **mode bit** (a flag in CPU hardware) decides which mode the CPU is in.

- 0 → Kernel Mode
- 1 → User Mode

👉 When a program runs in **user mode**, it can't directly access hardware (disk, memory, network). It must request the OS via **system calls**.

2. Which Processes/Threads Run Where?

- **User Mode**
 - All application processes (e.g., Chrome, Word, Java program).
 - **User threads** → Created by applications and managed by libraries or OS, run in user mode.
- **Kernel Mode**
 - **Kernel processes/threads** (e.g., scheduler, memory manager, device drivers).
 - System-level tasks like interrupt handling, context switching, I/O operations.

👉 Example:

- You run a **Java program** → runs in **user mode**.
- Your program does `readFile()` → a **system call** → CPU switches to **kernel mode** to talk to the disk driver → t

3. Difference Between User Mode and Kernel Mode Execution

Aspect	User Mode	Kernel Mode
Privilege Level	Restricted (cannot access hardware directly)	Full access (CPU, memory, devices)
Who runs here?	Applications, user processes, user threads	OS kernel, device drivers, kernel threads
Access to Memory	Only its own memory space	Can access any memory, including user process memory
Access to Hardware	No direct access → must use system calls	Direct access to hardware (disk, network, CPU instructions)
Instruction Set	Limited (safe subset)	Full instruction set, including privileged instructions
Failure impact	Crash affects only the application	Crash can bring down the entire system (Blue Screen of Death / Kernel Panic)

6. Real-Life Example

👉 Suppose you save a file in MS Word:

1. Word (User Mode) → calls `write()` system call.
2. CPU switches to Kernel Mode.
3. Kernel interacts with the **file system + disk driver**.
4. Data is written to the disk.
5. CPU switches back to User Mode, Word continues execution.

✅ Summary:

- Applications run in **user mode**, OS kernel and drivers run in **kernel mode**.
- User mode = **restricted, safe**, Kernel mode = **powerful, risky**.
- Safety is achieved by enforcing separation → only OS has direct hardware control.

4. Safety Concerns

- **Why separation exists?**
 - To protect system resources and ensure one bad program doesn't crash the whole OS.
 - If all code ran in kernel mode → any bug in user code could corrupt memory, devices, or the kernel itself.
- **In User Mode:**
 - If a process misbehaves (e.g., infinite loop, invalid memory access) → OS can terminate it safely.
 - Other processes remain unaffected.
- **In Kernel Mode:**
 - Bugs are dangerous!
 - Example: A faulty device driver can crash the entire OS (Blue Screen of Death in Windows, Kernel Panic in Linux).

5. Analogy

Think of it like:

- **User Mode** = Guest (you can only access certain areas in a hotel).
- **Kernel Mode** = Hotel Staff/Manager (has master key, can enter any room, control everything).